



TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES
MULTIPLATAFORMA

Departamento de Informática

PROYECTO

ExtreFit

Instrucciones de ejecución

Autor/es: Sergio Ramírez Pla, José Luis Moreno Torés

Curso Académico: 2º B DAM

Tutor Proyecto: Marina Preciado Agapito

Índice

1 Introducción	3
2 Objetivos	3
3 Tecnologías involucradas	5
En el desarrollo del proyecto ExtreFit se han utilizado diversas tecnologías que permiten construir una aplicación completa basada en una arquitectura cliente-servidor.	5
4 Proceso de desarrollo.....	6
4.1. Análisis	6
4.2. Desarrollo.....	8
4.3. Pruebas realizadas	9
5 Proceso de Despliegue.....	10
6 Propuesta de mejora o trabajos futuros	11
7 Bibliografía	12

1 Introducción

ExtreFit es una aplicación móvil multiplataforma desarrollada con el objetivo de ayudar a los usuarios a mejorar sus hábitos de entrenamiento y nutrición mediante una herramienta única, sencilla e intuitiva. Actualmente existen numerosas aplicaciones centradas únicamente en el ejercicio físico o en la alimentación, lo que obliga a los usuarios a utilizar varias herramientas para gestionar su progreso.

Ante esta situación, ExtreFit propone una solución integral que permite centralizar en una misma aplicación la planificación de entrenamientos, la gestión de dietas, el seguimiento del peso y la evolución física, así como la recepción de notificaciones motivacionales para fomentar la constancia y el cumplimiento de objetivos.

El proyecto ha sido desarrollado siguiendo una arquitectura cliente-servidor, utilizando Flutter para el desarrollo de la aplicación móvil, Spring Boot para la implementación del backend y Firebase Firestore como sistema de almacenamiento en la nube. Esta combinación de tecnologías proporciona una solución escalable, mantenible y preparada para futuras ampliaciones.

2 Objetivos

Debe quedar claro qué funcionalidad se pretende desarrollar y qué objetivos se pretenden conseguir.

El objetivo principal es desarrollar una aplicación completa que integre:

- Gestión de entrenamiento
- Gestión nutricional
- Persistencia de datos
- Sistema de autenticación

Además, se pretende garantizar que la aplicación sea escalable, mantenible y capaz de adaptarse a futuras mejoras, siguiendo una arquitectura modular basada en la separación entre frontend, backend y sistema de almacenamiento.

3 Tecnologías involucradas

En el desarrollo del proyecto ExtreFit se han utilizado diversas tecnologías que permiten construir una aplicación completa basada en una arquitectura cliente-servidor.

Frontend (Flutter):

Se ha utilizado Flutter como framework principal para el desarrollo de la aplicación móvil. Flutter permite crear aplicaciones multiplataforma utilizando una única base de código en Dart, facilitando el desarrollo y mantenimiento.

Backend (Spring Boot):

El backend ha sido desarrollado en Java utilizando Spring Boot, que permite la creación de APIs REST de forma sencilla y estructurada. Este componente se encarga de la lógica de negocio, validación de datos y comunicación con la base de datos.

Base de datos (Firebase Firestore):

Firestore es una base de datos NoSQL en la nube que permite almacenar información de forma flexible y escalable. Se utiliza para guardar usuarios, ejercicios, comidas y configuraciones.

Almacenamiento local (SQLite):

Se utiliza SQLite en el dispositivo para almacenar la sesión del usuario, evitando la necesidad de autenticación constante.

Otras tecnologías y librerías:

- HTTP: comunicación entre frontend y backend
- Crypto: cifrado de contraseñas
- Flutter Local Notifications: generación de notificaciones
- FL Chart: visualización de datos de progreso

4 Proceso de desarrollo

El desarrollo del sistema se ha llevado a cabo siguiendo una arquitectura cliente-servidor.

Flujo general de funcionamiento:

1. El usuario interactúa con la aplicación móvil desarrollada en Flutter.
2. El frontend envía peticiones HTTP al backend mediante una API REST.
3. El backend procesa la solicitud, aplica la lógica de negocio y accede a la base de datos.
4. Firestore devuelve los datos solicitados.
5. El backend envía la respuesta al frontend en formato JSON.
6. El frontend actualiza la interfaz con la información recibida.

Este modelo permite desacoplar la interfaz de usuario de la lógica del sistema, facilitando el mantenimiento y la escalabilidad.

Además, se han implementado funcionalidades como:

- Gestión de usuarios (registro y login)
- Recuperación de contraseña mediante código
- Consulta de ejercicios y comidas
- Registro de progreso del usuario
- Generación de notificaciones motivacionales

4.1. Análisis

En la fase de análisis se identificaron los principales requisitos funcionales del sistema, así como los distintos perfiles de usuario (usuario normal y administrador).

Se definieron los casos de uso principales, entre los que destacan:

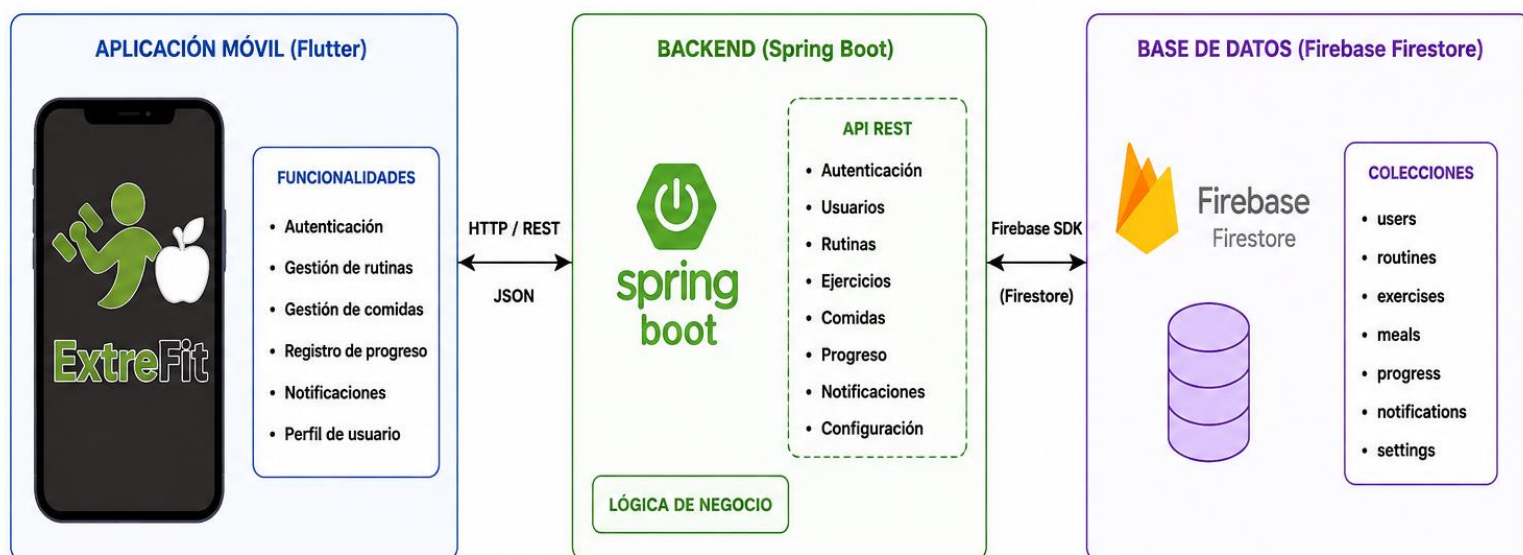
- Registro e inicio de sesión
- Gestión de rutinas de entrenamiento
- Planificación de dieta
- Seguimiento del progreso del usuario
- Recuperación de contraseña

Además, se diseñó la arquitectura general del sistema, estableciendo una estructura cliente-servidor en la que el frontend se comunica con el backend mediante una API REST.

A partir de este análisis se definieron también los modelos de datos necesarios y la estructura de almacenamiento en Firestore.

La base de datos se estructura en colecciones independientes en Firestore, lo que permite una gestión flexible y escalable de la información.

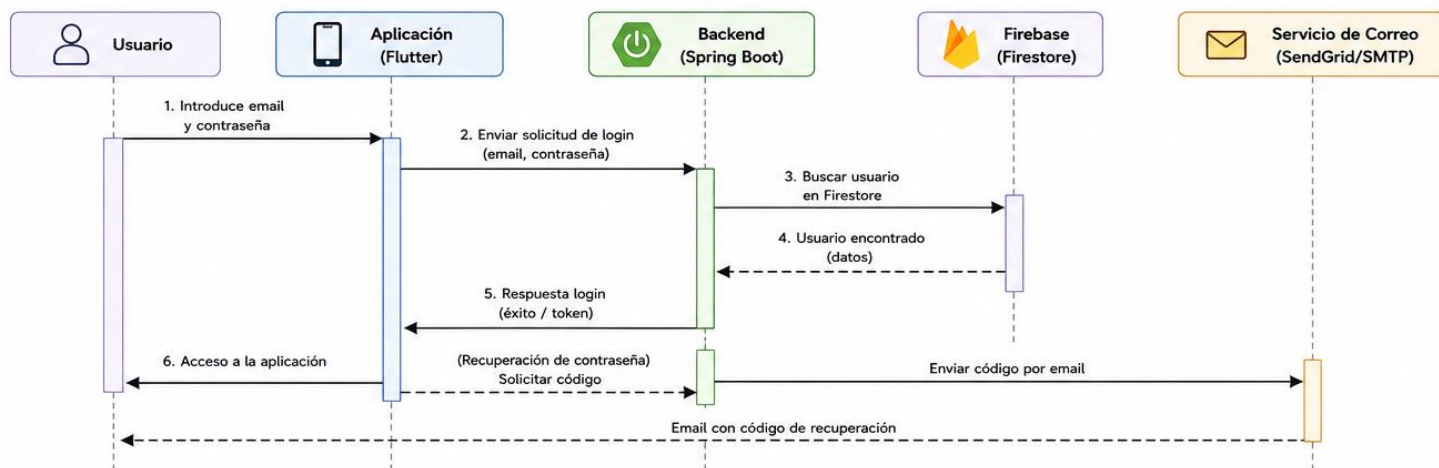
DIAGRAMA DE ARQUITECTURA DEL SISTEMA



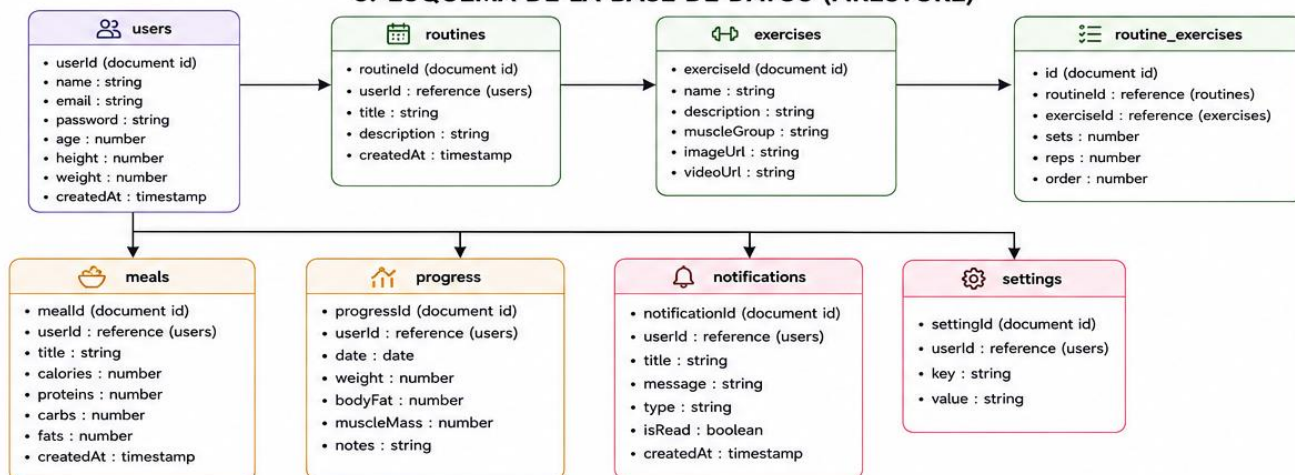
4.2. Desarrollo

El siguiente diagrama representa la interacción entre los distintos componentes del sistema durante el proceso de autenticación.

2. DIAGRAMA DE SECUENCIA: INICIO DE SESIÓN



3. ESQUEMA DE LA BASE DE DATOS (FIRESTORE)



El desarrollo de la aplicación se ha realizado siguiendo una arquitectura cliente-servidor, separando claramente la capa de presentación, la lógica de negocio y el almacenamiento de datos.

El proceso de funcionamiento del sistema es el siguiente:

1. El usuario interactúa con la aplicación móvil desarrollada en Flutter.
2. El frontend envía peticiones HTTP al backend mediante una API REST.
3. El backend procesa la solicitud, aplica la lógica de negocio y accede a Firestore.
4. Firestore devuelve los datos solicitados.
5. El backend responde al frontend en formato JSON.
6. El frontend actualiza la interfaz con la información recibida.

Durante el desarrollo se han implementado funcionalidades como la autenticación de usuarios, la gestión de rutinas y dietas, el registro de progreso y la generación de notificaciones.

Este flujo permite mantener una separación clara entre la capa de presentación, la lógica de negocio y el almacenamiento de datos, facilitando la mantenibilidad y evolución del sistema.

4.3. Pruebas realizadas

Durante el desarrollo del proyecto se han realizado diferentes pruebas funcionales para validar el correcto funcionamiento del sistema.

Prueba de registro de usuario:

Entrada: datos de usuario válidos

Resultado esperado: creación correcta del usuario en Firestore

Prueba de inicio de sesión:

Entrada: email y contraseña

Resultado esperado: acceso correcto a la aplicación

Prueba de recuperación de contraseña:

Entrada: email del usuario

Resultado esperado: envío de código y cambio de contraseña

Prueba de consulta de datos:

Entrada: petición de ejercicios o comidas

Resultado esperado: datos correctamente mostrados en la interfaz

Herramientas utilizadas:

- Pruebas manuales en dispositivo Android
- Logs del sistema (Logcat)
- Peticiones HTTP mediante herramientas de test

Las pruebas se han realizado principalmente de forma manual en dispositivos reales, verificando el comportamiento completo del sistema en condiciones reales de uso.

5 Proceso de Despliegue

El despliegue del sistema se ha realizado utilizando un entorno en la nube para el backend.

Backend:

Se despliega en Render, utilizando un contenedor basado en Java. El proceso incluye la compilación mediante Maven y la ejecución del archivo JAR generado.

Base de datos:

Firebase Firestore se utiliza como servicio en la nube, sin necesidad de despliegue local.

Frontend:

La aplicación se ejecuta directamente en dispositivos Android mediante Flutter.

Para ejecutar el sistema es necesario:

- Tener el backend en ejecución
- Disponer de conexión a internet
- Ejecutar la aplicación móvil

Este enfoque permite desacoplar el entorno de desarrollo del entorno de producción, facilitando la escalabilidad y el mantenimiento del sistema.

6 Propuesta de mejora o trabajos futuros

¿Qué mejoras se pueden introducir en el proyecto o qué ampliaciones se podrían hacer en un futuro sobre la base implementada?

Como posibles mejoras futuras se plantean:

- Implementar un sistema de autenticación más seguro (bcrypt)
- Dividir componentes grandes para mejorar la mantenibilidad
- Añadir pruebas automatizadas
- Incorporar nuevas funcionalidades como recomendaciones personalizadas
- Mejorar la interfaz de usuario
- Mejores animaciones
- Integración de un sistema de ranking
- Implementación de una pequeña red social

7 Bibliografía

- Documentación oficial de Flutter: <https://flutter.dev>
- Documentación oficial de Spring Boot: <https://spring.io>
- Firebase Firestore: <https://firebase.google.com>
- Maven: <https://maven.apache.org>
- Render: <https://render.com/>
- GitHub: <https://github.com/>